卢卡斯定理（Lucas Theorem）

卢卡斯定理(Lucas Theorem)用于计算组合数取模，即求 $C_n^r \bmod m$，其中 m 为素数。

概念回顾

求解一般形式的同余方程 $ax \equiv b \pmod{m}$，需要用到逆(Inverse)。

逆的概念

给定整数 a，且满足 $\gcd(a, m) = 1$，称 $ax \equiv 1 \pmod{m}$ 的一个解为 **a 模 m 的逆**，记为 a^{-1}。

例如，$8x \equiv 1 \pmod{31}$，有一个解是 $x = 4$，4 是 **8 模 31 的逆**。所有解，如 35、66 等，都是 8 模 31 的逆。

可以借助方程理解逆的概念，$8x \equiv 1 \pmod{31}$ 即方程 $8x + 31y = 1$，$x = 4$ 是 8 模 31 的逆，$4 \times 8 - 1$ 能整除 31。

从逆的要求 $\gcd(a, m) = 1$ 可以看出，做竞赛题时，**模 m 最好是一个大于 a 的素数**，能保证 $\gcd(a, m) = 1$。如果模 m 不大于 a 时的可以用我们要提到的卢卡斯定理解决。

费马小定理求单个逆

有多种方法可以求逆，比如使用扩展欧几里得算法求单个逆，使用费马小定理求单个逆，或者使用递推求多个逆。这里我们回顾费马小定理。

费马小定理 设 n 是素数，a 是正整数且与 n 互素，有 $a^{n-1} \equiv 1 \pmod{n}$。

$a \cdot a^{n-2} \equiv 1 \pmod{n}$，那么 $a^{n-2} \bmod n$ **就是 a 模 n 的逆**。计算需要用到快速幂。

用逆求解同余方程

逆有什么用？如果有 a 模 m 的一个逆，可以用来求解形如 $ax \equiv b \pmod{m}$ 的任何同余方程。

记 a^{-1} 是 a 的一个逆，有 $a^{-1}a \equiv 1 \pmod{m}$。在 $ax \equiv b \pmod{m}$ 的两边同时乘以 a^{-1}，得到 $a^{-1}ax \equiv a^{-1}b \pmod{m}$，即 $x \equiv a^{-1}b \pmod{m}$。

例如，为了求出 $8x \equiv 22 \pmod{31}$ 的解，可以两边乘以 4，4 是 8 模 31 的一个逆，得

$4 \times 8x = 4 \times 22 \pmod{31}$，因此 $x \equiv 88 \pmod{31} \equiv 26 \pmod{31}$。

定理 设 p 是素数，正整数 a 是**其自身模 p 的逆**，当且仅当 $a \equiv 1 \pmod{p}$ 或 $a \equiv -1 \pmod{p}$。

证明 若 $a \equiv 1 \pmod{p}$ 或 $a \equiv -1 \pmod{p}$，有 $a^2 \equiv 1 \pmod{p}$，所以 a 是其自身模 p 的逆。反过来也成立。

逆与除法取模

逆的一个重要应用是求除法的模。求 $(a/b) \bmod m$，即 a 除以 b，然后对 m 取模。这里 a 和 b 都是很大的数，如 $a = n!$，容易溢出，导致取模出错。用逆可以避免除法计算，设 b 的逆元是 b^{-1}，有

$$(a/b) \bmod m = ((a/b) \bmod m)\left(bb^{-1} \bmod m\right) = (a/b \times bb^{-1}) \bmod m = (ab^{-1}) \bmod m$$

经过上述推导，除法的模运算转换为乘法模运算，即

$(a/b) \bmod m = \boldsymbol{(ab^{-1})} \bmod \boldsymbol{m} = (a \bmod m)(b^{-1} \bmod m) \bmod m$

卢卡斯定理和证明

组合数的计算可以用逆，但是也有局限。回顾逆的定义：给定整数 a，满足 $\gcd(a,m)=1$，称 $ax \equiv 1 \pmod{m}$ 的一个解为 a 模 m 的逆，记为 a^{-1}。从要求 $\gcd(a,m)=1$ 可以看出，模 m 最好是一个大于 a 的素数，才能保证 $\gcd(a,m)=1$。但是，之前我们利用逆计算组合数：
$C_n^r \bmod m = (n! \bmod m) \cdot ((r!)^{-1} \bmod m) \cdot (((n-r)!)^{-1} \bmod m) \bmod m$

如果模 m 比 n 小，就不能保证 n 和 $n-r$ 的逆元存在。此时不能用逆计算，但是如果用递推公式计算，复杂度为 $O(n^2)$，比较低效。卢卡斯定理仍能高效率地处理这种情况。

卢卡斯定理 对于非负整数 n、r 和素数 m，有
$C_n^r \equiv \prod_{i=0}^{k} C_{n_i}^{r_i} \pmod{m}$

或者写成
$C_n^r \bmod m = \prod_{i=0}^{k} C_{n_i}^{r_i} \bmod m$

其中，$n = n_k m^k + \cdots + n_1 m + n_0$ 和 $r = r_k m^k + \cdots + r_1 m + r_0$ 是 n 和 r 的 m **进制展开**，也就是把 n 和 r 表示为 m 进制数，对 m 进制下的每位分别计算组合数，最后乘起来。例如，计算 $C_{17}^8 \bmod 7$，$17 = 2 \times 7^1 + 3 \times 7^0$，$8 = 1 \times 7^1 + 1 \times 7^0$，
$C_{17}^8 \bmod 7 = C_2^1 \times C_3^1 \bmod 7 = 6 \bmod 7 = 6$。

注意，在这个公式中有可能 $n_i < r_i$，此时规定 $C_{n_i}^{r_i} = 0$。如果有一个 $n_i < r_i$，就有
$C_n^r \bmod m = 0$。

编程时用卢卡斯定理的另一种表达：
$C_n^r \equiv \mathbf{C}_{\mathbf{n} \bmod \mathbf{m}}^{\mathbf{r} \bmod \mathbf{m}} \cdot \mathbf{C}_{\mathbf{n/m}}^{\mathbf{r/m}} \pmod{m}$

下面证明这个公式。

证明 对于素数 m，$r \in (0,m)$，有
$C_m^r = \frac{m!}{r!\,(m-r)!} = \frac{(m-1)!}{(r-1)!(m-r)!} \times \frac{m}{r} = C_{m-1}^{r-1} \times \frac{m}{r} \equiv 0 \pmod{m}$

将其代入二项式定理的展开式，得
$(1+x)^m = \sum_{r=0}^{m} C_m^r x^r = 1 + \sum_{r=1}^{m-1} C_m^r x^r + x^m \equiv 1 + x^m \pmod{m}$

下面推导卢卡斯定理。令 $n = sm + a$，有
$(1+x)^n = (1+x)^{sm+a} = (1+x)^{sm}(1+x)^a \equiv (1+x^m)^s(1+x)^a$
$\equiv \left(\sum_{i=0}^{s} C_s^i x^{im}\right) \times \left(\sum_{j=0}^{a} C_a^j x^j\right) \pmod{m}$

又根据二项式定理 $(1+x)^n = \sum_{r=0}^{n} C_n^r x^r$，与上式对比得
$\sum_{r=0}^{n} C_n^r x^r \equiv \left(\sum_{i=0}^{s} C_s^i x^{im}\right) \times \left(\sum_{j=0}^{a} C_a^j x^j\right) \pmod{m}$

对比两边第 x^r 次项的系数，根据 $r = im + j$，$i = r/m$，$j = r \bmod m$，$a = n \bmod m$，$s = n/m$，得
$C_n^r \equiv C_s^i \times C_a^j \pmod{m} \equiv \mathbf{C}_{\mathbf{n/m}}^{\mathbf{r/m}} \times \mathbf{C}_{\mathbf{n} \bmod \mathbf{m}}^{\mathbf{r} \bmod \mathbf{m}} \pmod{m}$

注意，公式分为两部分：
(1) $C_{n \bmod m}^{r \bmod m} \pmod{m}$，因为 $r \bmod m$ 和 $n \bmod m$ 都比 m 小，仍然可以使用“用逆直接计算二项式公式”来求解；
(2) $\mathbf{C}_{\mathbf{n/m}}^{\mathbf{r/m}}$，继续用卢卡斯定理展开，用递归实现。

模板题

下面给出一道卢卡斯定理的模板题，实现了上述公式的两部分计算。

卢卡斯定理（洛谷 P3807）

问题描述：给定整数 a、b、m，求 $C_{a+b}^{a} \bmod m$。保证 m 为素数。

输入：第 1 行输入一个整数 T，表示测试数量；后面 T 行中，每行输入 3 个整数 a、b、m。

输出：共 T 行，每行输出一个整数，表示答案。

数据范围：$0 \le a, b, m \le 100000$，$1 \le T \le 10$。

下面代码的复杂度如何？Lucas()函数用递归实现辗转相除 n/mod，当 mod 是一个较大的素数时，Lucas()函数只需递归很少次。计算主要花在预计算阶乘上，复杂度为 $O(n)$。

```
#include<bits/stdc++.h>
using namespace std;
const int N = 100010;
typedef long long ll;
ll fac[N];                          //预计算阶乘，取模
ll fastPow(ll a, ll n, ll m){       //标准快速幂
    ll ans = 1;
    a %= m;                         //防止下面的ans*a越界
    while(n) {
        if(n & 1)   ans = (ans*a) % m;
        a = (a*a) % m;
        n >>= 1;
    }
    return ans;
}
ll inverse(ll a,int m){ return fastPow(fac[a],m-2,m); } //用费马小定理计算逆
ll C(ll n,ll r,int m){          //用逆计算C(n mod m, r mod m) mod m
    if(r>n)return 0;
    return ((fac[n] * inverse(r,m))%m * inverse(n-r,m)%m);
}
ll Lucas(ll n,ll r,int m){      //用递归计算C(n, r) mod m
    if(r==0) return 1;
    return C(n%m,r%m,m) * Lucas(n/m,r/m,m)%m;           //分两部分计算
}
int main(){
    int T; cin >> T;
    while(T--){
        int a,b,m; cin >>a>>b>>m;
        fac[0] = 1;
        for(int i=1;i<=m;i++)   fac[i]=(fac[i-1]*i)%m;  //预计算阶乘，取模
        cout << Lucas(a+b,a,m) << endl;
    }
}
```